

The Normal Equations Strike Back: the $n < p$ Case

Albert Dorador
trustalgorithm.dev@gmail.com

December 8, 2025

Abstract

Classical numerical wisdom states that when $n < p$ one should never form the normal equations $X^T X \beta = X^T y$ for ordinary least squares because the $p \times p$ Gram matrix is both slower and less stable than SVD-based solvers. Using a small data-driven ridge penalty $\lambda = 10^{-9} \times \text{avg}(\|X_{:,j}\|_F^2) \approx 10^{-6}$ to enforce strict positive-definiteness, we show that ridge-regularized normal-equation solvers remain faster than SVD-based solvers like `scipy.linalg.lstsq` in the underdetermined regime far beyond where textbooks expect them to fail. Only when $p \gtrsim 10000$ or $p/n \gtrsim 20$ does the cost of forming and factoring $X^T X$ finally dominate. Solving the normal equations is also far more stable numerically than many expect: coefficient disagreement is at most 10^{-7} for the LU/Cholesky-based approaches. The 50-year taboo against the normal equations is obsolete even in many high-dimensional settings.

1 Introduction

In a companion paper [4] we showed that in the tabular regime $n \geq p$ forming the normal equations to solve ordinary least squares (OLS) problems is perfectly safe and 3–18× faster than the often-recommended SVD-based approaches. The standard counter-argument has always been: “but when $n < p$ you *must* use SVD decomposition because forming the $p \times p$ Gram matrix is prohibitively expensive”.

It is true that forming the normal equations is a theoretically inferior way of solving OLS problems when $n < p$, as both computing the Gram matrix and factoring it are expensive operations, requiring $\mathcal{O}(np^2)$ and $\mathcal{O}(p^3)$ operations, respectively. In such contexts, an SVD-based approach is theoretically superior in terms of computational complexity: $\mathcal{O}(np \min(n, p))$, which reduces to $\mathcal{O}(n^2 p)$ in the $n < p$ case.

However, this short follow-up demonstrates that – with a negligible ridge penalty – the normal equations remain competitive or superior in the underdetermined regime up to $p \gtrsim 10000$ or $p/n \geq 20$, i.e. well into what is traditionally considered the “high-dimensional” domain (e.g. genomics or text embeddings).

2 Methods

We compare four standard approaches on synthetic dense data without an intercept term:

1. SVD-based, minimum-norm solution (`scipy.linalg.lstsq`)
2. Normal equations (ridge-regularized) with Cholesky decomposition (`scipy.linalg.cholesky`)
3. Normal equations (ridge-regularized) with LU solver (`np.linalg.solve`)

4. Normal equations (ridge-regularized) with Gram inverse (`np.linalg.inv`)

A small, data-driven ridge penalty [5] is added to enforce strict positive-definiteness:

$$\lambda = 10^{-9} \times \frac{1}{p} \text{tr}(X^T X) = 10^{-9} \times \text{avg}(\|X_{:,j}\|_F^2) \text{ (average squared column Frobenius norm)}.$$

which corresponds to a ridge penalty in the order of 10^{-6} with our standard Gaussian-generated data. For the LU/Cholesky-based approaches, this data-driven choice of ridge penalty induces coefficient disagreement of at most order 10^{-7} versus the exact minimum-norm solution. Perhaps surprisingly, even with the approach that explicitly inverts the Gram matrix, coefficient disagreement is still small ($\lesssim 10^{-5}$).

Besides the introduction of a ridge penalty as described above, we follow the same methodology as in our companion paper [4] discussing the $n \geq p$ regime.

Design matrices X and response vectors y were generated as i.i.d. standard Gaussian entries, which is common practice in the literature and avoids arbitrary choices of correlation structures and signal-to-noise ratios (the latter having no impact in solving times or stability).

100 individually timed per-solve repetitions were carried out per method and scenario, allowing us to calculate error bands around the mean recorded times. We do not assume normality of the recorded times and instead consider two standard errors (which, in the absence of normality, may not correspond to a traditional 95% confidence interval but in practice it is likely in its vicinity). These error bands provide a sense of ‘significance’ that can be valuable when analyzing the results.

Both absolute times as well as relative times to SciPy’s `lstsq` are reported. The choice of baseline obeys to the popularity of the `scipy.linalg.lstsq` routine to solve OLS problems, being the method of choice of the mainstream `scikit-learn` machine learning library, explicitly stated for instance in `sklearn.linear_model.LinearRegression`.

A seed (123) was fixed to guarantee partial reproducibility (e.g. condition numbers). Exact, full reproducibility is impossible to guarantee as runtimes (especially at microsecond scale) are sensitive to factors that can’t be easily controlled such as concurrent computer processes. The code to obtain our results is accessible at <https://github.com/adctrust-ai/trust-free>.

Experiments were run on Python 3.11.13, NumPy 1.26.4, SciPy 1.16.3, on an Apple M4.

3 Results

Wall-clock times for 100 repetitions per method and scenario are reported in Table 1. Figure 1 visualizes performance relative to SciPy’s `lstsq`.

These results show, perhaps surprisingly, that, in the $n < p$ regime, forming and solving the ridge-regularized normal equations in practice can be even faster than the standard SVD-based approach except in extreme settings ($p \gtrsim 10000$ or $p/n \gtrsim 20$), especially if an approach based on the Cholesky decomposition is used. Absolute coefficient disagreement of this approach versus the standard SVD-based approach delivering minimum-norm solutions is always below $\sim 10^{-7}$.

Despite a better-than-expected performance, inverting the Gram matrix is discouraged, as it is both slower and less stable than other alternatives (coefficient disagreement $\lesssim 10^{-5}$).

4 Discussion and Recommendation

Our results show that in OLS problems with $n < p$, ridge-regularized normal-equation solvers dominate the book-standard SVD approach (`scipy.linalg.lstsq`) up to $p \gtrsim 10000$ or $p/n \gtrsim 20$,

Table 1: Mean time (ms) $\pm$ 2 std. errors across 100 repetitions. Results in bold show the fastest method(s) up to error-band precision.

n	p	SciPy’s lstsq	N.E. + Chol	N.E. + LU	N.E. + inv
10	20	0.028 ± 0.004	0.019 ± 0.003	0.147 ± 0.086	0.308 ± 0.151
10	200	0.070 ± 0.012	0.174 ± 0.014	0.364 ± 0.032	0.582 ± 0.038
10	1000	0.178 ± 0.016	5.540 ± 0.262	8.124 ± 0.393	20.100 ± 0.524
20	100	0.097 ± 0.006	0.080 ± 0.021	0.174 ± 0.018	0.174 ± 0.039
50	200	0.564 ± 0.031	0.183 ± 0.010	0.343 ± 0.009	0.560 ± 0.016
100	500	2.695 ± 0.148	1.294 ± 0.074	1.967 ± 0.133	4.659 ± 0.317
200	1000	8.930 ± 0.401	7.541 ± 0.319	10.001 ± 0.398	24.407 ± 0.496
500	2000	48.99 ± 1.20	41.27 ± 1.10	56.92 ± 1.16	147.07 ± 1.51
1000	2000	151.1 ± 3.7	51.5 ± 1.0	66.2 ± 0.6	174.6 ± 1.3
2000	10000	1748 ± 7	3254 ± 34	5510 ± 66	17677 ± 258

i.e. well into the regime traditionally considered “high-dimensional”. Only after said thresholds does forming and solving the ridge-regularized normal equations become too expensive in front of the more efficient SVD-based solutions. The transition point may depend mildly on hardware (memory bandwidth vs floating-point throughput) but is far higher than classical textbook intuition suggests [6, 1, 5].

Practitioners facing $n < p$ OLS problems without extreme n vs p imbalance and a few thousand variables or fewer now have a reliable, faster alternative: form the Gram matrix, add a negligible relative ridge penalty, and solve the triangular systems produced by Cholesky. For $n \ll p$ the traditional SVD-based path remains correct.

This finding can make a material difference in applications needing to solve repeated OLS problems, potentially in the high-dimensional regime, like linear model tree fitting [3] or VIF / ghost variable calculations [2].

The normal equations are no longer forbidden anywhere except the extreme $n \ll p$ tail.

References

- [1] Cheney, W. and Kincaid, D. (2008). *Numerical Mathematics and Computing* (6th ed.). Thomson Higher Education, Belmont.
- [2] Delicado, P. and Pena, D. (2023). *Understanding complex predictive models with ghost variables*, TEST 32: 107–145.
- [3] Dorador, A. (2025). *TRUST: Transparent, Robust and Ultra-Sparse Trees*, PRICAI 2025. Lecture Notes in Artificial Intelligence, Springer Nature (forthcoming). arXiv: 2506.15791.
- [4] Dorador, A. (2025). *The Normal Equations Are Normally Faster: Benchmarking Ordinary Least Squares Solvers for $n \geq p$* , SSRN: <http://dx.doi.org/10.2139/ssrn.5811842>.
- [5] Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- [6] Lawson, C.L. and Hanson, R.J. (1974). *Solving Least Squares Problems*. Prentice-Hall.

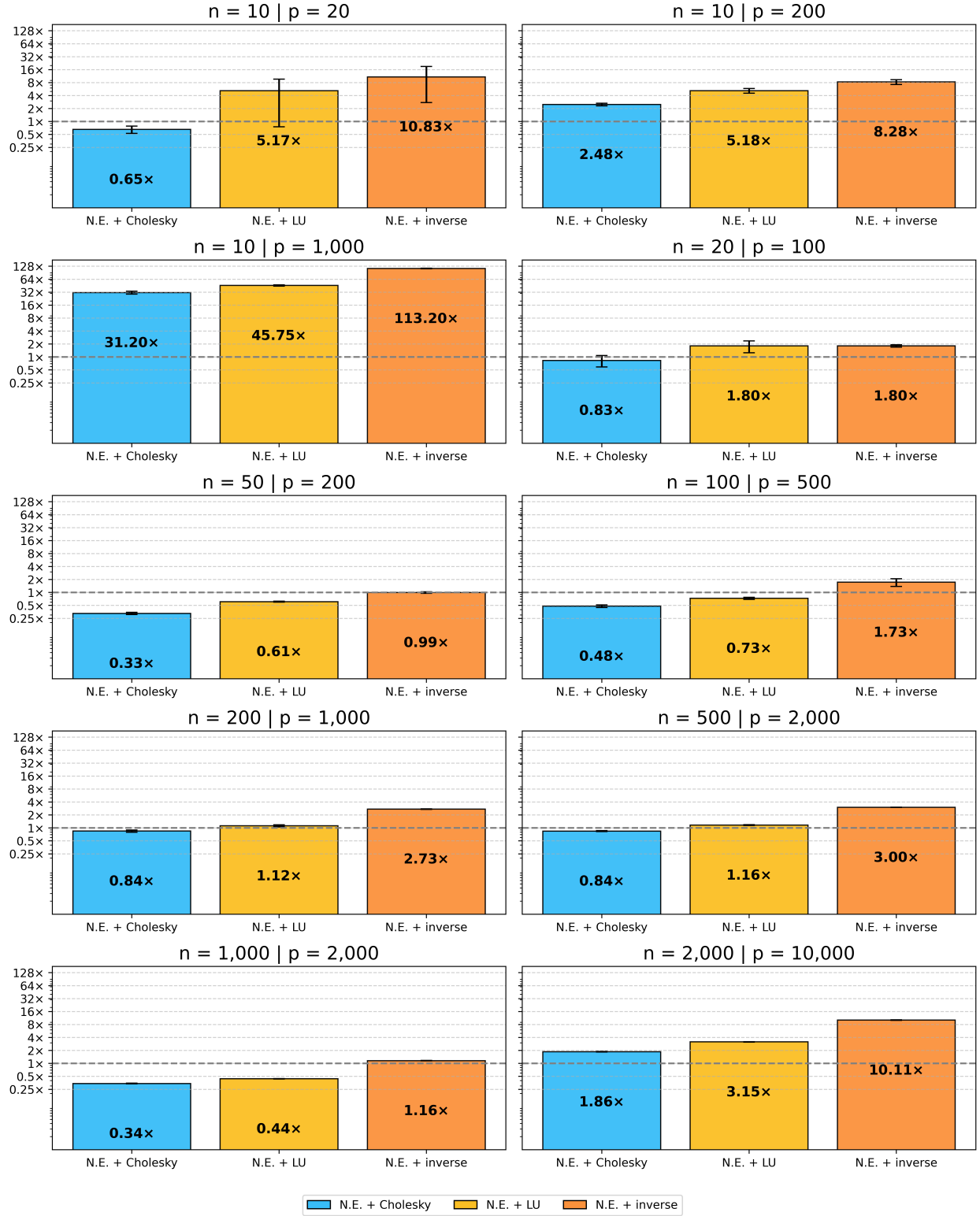


Figure 1: Runtime relative to SciPy's lstsq with 2×SE error bands (logarithmic scale)